

Smart Local MPLPB

A Retrieval-Bounded Front End for a Local Web

Mitchell D. McPhetridge

Independent Researcher

Field	Value
Document ID	MPLPB-SMART-011
Category	Specification / Reference Implementation
Updated	2026-08-12T00:00Z v1
Status	current
Scope	The front-end layer that queries an MPLPB local web, routes questions by declared scope, cites retrieved pages with provenance, and writes taught answers back as documents
When to use	Building or deploying a front end over a local web; deciding what a retrieval layer is allowed to answer; reviewing the mode controller as implemented rather than as specified
Supersedes	—

Related. *MPLPB as a Local Web* (MPLPB-LOCAL-008 v4) — the corpus this front end reads. *A Modular Mode-Switching Architecture for a Mythic-Logic Companion* (MPLPB-MODE-008) — the mode controller. *Separate, Allowed to Chat, and Overridden by Scope* (MPLPB-SEP-007) — the precedence rule. *B.A.S.I.C. AI: A Walk Down Memory Lane with Code and Curiosity* (2025) — the loop.

Back to. [Main Index](#) > [Specification / Architecture Sub-Index](#)

Abstract

The MPLPB local-web specification describes a bounded knowledge corpus as an actual website: plain HTML files in a directory, connected by relative links, starting at `index.html`. It specifies the file layout, the page metadata, the crawl algorithm, the three ingestion paths, and eight structural checks. What it does not specify is what happens at the other end — what a program that consults such a corpus is permitted to do with what it retrieves, and what it must do when it retrieves nothing.

This paper describes Smart Local MPLPB, a reference front end that answers that question in code. It is a standard-library Python package providing a chat console, a browser view, and a library API over a local web. Its defining property is negative: it cannot answer from anywhere except a page the crawler found under the corpus root. Nothing matched produces *not in this corpus*, never a plausible paragraph from elsewhere.

Four commitments follow from that one. Retrieval carries provenance that cannot be separated from the text, so that a local page is never cited as a published one. Routing respects declared scope, so that two spokes with something to say about the same question produce a request to narrow rather than a blended answer. Mode switching assigns authority in advance, so that a safety-adjacent topic reaches the careful mode before any other trigger can claim it. And teaching — the *tell me and I will remember* loop inherited from a 1960s-language toy chatbot — writes a validating, superseding, cited document rather than appending a line to a file.

The implementation is 52 tests, no dependencies, and no network. The ablation that would establish whether any of this beats a flat directory of the same files has not been run, and §14 states it alongside four other falsifiers.

1. The Gap This Fills

MPLPB-LOCAL-008 defines a corpus and the discipline that keeps it honest. It is careful about the things a corpus can be wrong about: a broken link, an orphan page, a duplicate document ID, a retired page in the wrong place, a timestamp that cannot be ordered. Its failure-mode index runs to eleven entries, and the validator catches eight of them mechanically.

But three of those eleven failures are not defects in the corpus at all. They are defects in whatever is reading it.

- **FM-L7, retrieval without provenance.** The search layer returns a fragment

with no document identity attached. The fragment may be perfectly accurate and still be unusable, because nothing downstream can tell whether it is current.

- **FM-L8, model bypass.** The reader answers a corpus-specific question from

its own prior knowledge instead of a retrieved page. The corpus was well-formed, crawlable, and irrelevant.

- **FM-L10, local/public confusion.** A local document is cited as though it

were published, inventing an authority the corpus never claimed.

No amount of structural validation prevents any of these. A corpus can pass all eight checks and be read by a program that ignores it. The specification says this plainly and then, correctly, stops: the reader was out of scope.

Smart Local MPLPB is the reader, written to make those three failures structurally difficult rather than merely discouraged.

2. What a Front End Is Allowed to Do

The design rests on one asymmetry. A front end that occasionally says *I don't have that* is mildly annoying. A front end that occasionally invents a confident answer that reads exactly like a retrieved one is corrosive, because the user cannot tell the two apart, and after a few undetected instances the provenance on every other

answer stops meaning anything.

The two failures are not symmetric, so the design is not balanced between them. Every mode in the shipped configuration carries `answer_without_retrieval: false`. There is no mode that improvises. When retrieval returns nothing, the console says so, lists the spokes that do exist with their declared scopes, and — if the running mode permits — offers to be taught.

This is the whole argument for the package. Everything below is machinery in service of it.

3. Four Layers

The local-web specification separates three layers: the corpus, the crawler and index, and the reasoning that queries them. This implementation keeps that separation and splits the third, because *deciding where an answer comes from* and *writing the answer* are different jobs with different failure modes.

Layer	Modules	Owns
1. Corpus	<code>page.py</code> , <code>crawl.py</code> , <code>validate.py</code>	What a page is, how to enumerate pages, whether the corpus is well-formed
2. Retrieval	<code>index.py</code> , <code>cite.py</code>	Finding pages, and the provenance that travels with them
3. Decision	<code>router.py</code> , <code>modes.py</code>	Which page should answer, and what the answer is permitted to be
4. Presentation	<code>console.py</code> , <code>server.py</code> , <code>notebook.py</code>	How it reaches a person, and how a person writes back

The split matters because layer 3 produces a data structure, not prose. The router returns a routing decision — kind, primary page, spokes touched, declared owner, related pages, path back — and the console renders it. A different renderer can sit on the same decision: a template, a person, or a language model. In every case the provenance rules were already enforced one layer down, where they cannot be talked out of.

4. Retrieval Discipline

Retrieval is a SQLite FTS5 index built in memory from crawl records, with a LIKE-based ranker as a fallback when the host interpreter's `sqlite3` was compiled without FTS5. A corpus should not stop being searchable because of how somebody's Python was built.

Three rules shape what comes back.

Narrow before broad. The first pass requires every content term. Only if that returns nothing does a second pass accept any term. Requiring all terms first is what stops a question dragging in a page that merely shares one common word with it — which is precisely how routing ends up calling two unrelated spokes a tie.

A shared ordinary word is not a match. The broad pass only earns a result if the question contained a term that is distinctive in this corpus, defined as appearing in at most half the pages. Without this guard, *what does the contract say about liability* retrieves a page about document supersession, because that page happens to

contain the word "say". The result looks like retrieval and is not, which is worse than no answer.

Current by default; retired on request. Default retrieval filters to status = current, per §7.1. Retired pages remain reachable by explicit status filter or by document ID, and every returned record surfaces its status. This is what preserves the corpus's ability to answer *what did this document used to say* — the capability FM-L11 exists to protect, and the one that is invisible to every other check.

5. Provenance

Every hit carries eleven fields, of which six are provenance: document ID, path, version, status, spoke, and substrate. They are attributes of the hit object, not decorations added at render time, so there is no code path that produces text without them.

substrate is always local. Every citation says so:

```
[LOCAL-SPEC-001 · spec/link_discipline.html · v2 · local · current]
```

This is deliberately slightly awkward to read. FM-L10 is a quiet failure — a local note cited as a published source reads better than the truth, which is exactly the problem. The citation format makes the honest version the default version and gives no shorter form that omits it.

Retired hits carry a visible flag and a line stating that their current replacement, if any, is the one to act on. A retrieved retired page is a correct answer to a provenance question and a wrong answer to an operational one, and only the reader knows which was asked.

6. Routing and Precedence

The boot block specifies routing in four clauses: broad requests go to the relevant Sub-Index, specific requests to the best operational page, uncertain requests back to the Main Index with a request to narrow, and every answer names one to three related pages and the path back. The router implements all four and adds a fifth case the boot block does not name.

Separate, Allowed to Chat, and Overridden by Scope establishes that for any question, the MPLPB whose declared scope owns that question overrides any out-of-scope instance that also speaks to it. Authority is local to the question and decided by declared scope, not by size, age, confidence, or link count. That paper generalizes the priority-override pattern from safety-versus-narrative to domain-versus-domain.

Made executable, this becomes a scoring problem. Each spoke accumulates rank-decayed weight from its hits, plus a bonus when its own declared scope names the query's terms — declared scope is weighted because declared scope is what precedence is decided by. A spoke owns the question when it leads the runner-up by 1.5×.

Below that margin, the router returns ambiguous. The console names the spokes it touched, quotes each declared scope, and asks the user to narrow. It does not answer. Two MPLPBs that always defer to whoever spoke last are not separate; they are one blurred instance wearing two names, and an answer silently assembled from both is the fast version of the same collapse.

Two smaller rules keep this from misfiring. Only a directory that declares a scope in its Sub-Index can contend for ownership — the Main Index and `_log/` hold pages but claim no domain, so the revision log never wins an argument about physics. And a page is preferred over its Sub-Index when the question names something the page's own when `-to-use` triggers declare, which is what the trigger field is for.

The 1.5× margin is a chosen constant, not a derived one. It is the most arbitrary number in the package and the first thing worth tuning against a real corpus. §14.3 states the test.

7. The Mode Controller

The mode architecture from MPLPB-MODE-008 arrives almost unchanged: four modes, explicit override phrases, implicit topic triggers, and `hard_reality_mode` at priority 100.

Resolution order is fixed: an explicit override phrase, then a hard-reality trigger, then any other implicit trigger, then the mode already running. The point of assigning priority in advance is that the system is never in the position of guessing which behaviour to believe after a conflict has already arisen. Hard reality is the mode that overrides, so it is the one decided first, and it cannot be talked out of by a later phrase in the same line. *Walk me through this contract clause* contains a teaching trigger and a hard-reality trigger; it lands in hard reality.

Three departures from the paper are worth naming.

The default is `strict_tool_mode`, not `mythic_gm_mode`. The paper defaults to the mythic mode; the MPLPB boot block defaults to strict tool. A corpus front end follows the boot block. Configurable either way.

Every configuration key changes observable behaviour. The paper's engine JSON carries module toggles this front end has no implementation for. A config key that changes nothing is a claim the software does not honour, so those keys are absent rather than present and inert.

Hard reality declines to write. `teach_allowed: false`. Legal, medical-adjacent, contractual, and safety topics should not enter a corpus as something a session typed in and a later session reads back as a document, carrying a document ID and a timestamp that make it look considered. The mode that exists to be careful is the mode that refuses to record. When it triggers, the console names the trigger word, states that the corpus is not professional advice, and drops every overlay.

One addition the paper did not need: triggers are classified as instructions or as subjects. *Walk me through* and *explain* describe how to answer, and are stripped from the retrieval query. *Contract* and *dosage* are what the question is about, and are kept. Without the distinction, asking the console to walk you through supersession retrieves every page containing the word "walk".

The mythic mode is retained in full, with one constraint carried over from the paper's own identity clause: the symbolic frame decorates the presentation of a retrieved page, never its content, and never speaks as though the console were an entity. It remains a tool, and says so.

8. Teaching: Memory That Becomes a Document

Twelve lines of BASIC from 2025:

```

60 INPUT Q$
80 OPEN "data.txt" FOR INPUT AS #2
120 IF INSTR(DATA$, Q$) THEN PRINT "I remember! "; ...
190 IF ResponseFound = 0 THEN
200 PRINT "I don't know the answer to that. What should I say next time?"
220 OPEN "data.txt" FOR APPEND AS #1
230 PRINT #1, Q$; ":"; A$

```

The loop is right and is kept without alteration: check what you know, and when you don't know, ask, and remember. The claim that program made for itself was modest and correct — memory and association are powerful, and even the simplest system can appear smart if it remembers what you told it last time.

What could not survive the move to a local web was `data.txt`. An appended line has no document ID, no scope, no timestamp, no status, and no way to be superseded. The crawler cannot see it. It is the flat-file form of an orphan page, and a corpus that accumulates them silently develops a second, invisible body of knowledge with none of the discipline that governs the first.

So the taught answer becomes a page. `notebook.teach()` writes HTML with the full required metadata, assigns the next `LOCAL-NOTE-nnnn` identifier, lists it in the notebook Sub-Index, appends a line to the revision log, and links the spoke from the Main Index if it is not already there. The result passes all eight structural checks, is found by the graph crawl on the next pass, is retrievable, and is citable with provenance. Its face states what it is: *a record of what someone said, not a verified claim*.

Revision is supersession, never overwriting. `notebook.revise()` performs all five steps together, because any one of them omitted leaves the corpus disagreeing with itself: the old file moves to `_log/superseded/`, its status flips to `retired`, it is removed from the Sub-Index, the replacement names it in `supersedes`, and the change is logged. The retired page's relative links are rewritten for its new depth, so it still resolves. It is never deleted.

The consequence is the one worth having. `:history LOCAL-NOTE-0001` returns both versions, each with its own timestamp and status. A conversational correction — *no, it's cone 06 to 04 depending on the clay body* — leaves a dated record of what was believed before and what replaced it. The BASIC program overwrote nothing and remembered everything; this remembers everything and can also say when it changed its mind.

9. Validation as a Runtime Facility

The eight checks are not only a build step. `:validate` runs them inside a session, and the browser front end exposes them at `/_report`. This is a consequence of the console being able to write: a front end that adds pages should be able to prove, at any moment, that the pages it added did not break anything.

Findings are structured objects — check number, path, message, and the FM-L code where the specification assigns one — rather than printed strings, so a caller can act on them. The CLI formats them at the edge. `validate` and `crawl` exit 1 on any failure, so both drop into CI unchanged; the included workflow validates the example corpus, crawls it, runs the tests on three Python versions, and scaffolds a fresh corpus to confirm that a new one validates before it contains anything.

10. Two Front Ends

The console is the primary interface: the loop, the modes, the teaching, and four introspection commands that exist because a retrieval system that cannot explain itself is difficult to trust. `:where` reports the page the last answer came from in full, with its declared scope and the notice that it is a local document. `:why` reports the routing decision with per-spoke scores and the margin rule. `:history` walks a supersession chain. `:status` any opens retrieval to retired pages.

The browser view exists because the corpus is already a website — opening `index.html` in a browser works with no server at all. The server adds the one thing a `file://` browse cannot do: search across pages, with each result carrying its identity, and a structural report. It is read-only and bound to `localhost`. Teaching happens in the console, where a person is present to be asked what they mean.

11. Failure Modes Addressed

Code	Failure	How it is made difficult
FM-L1	Broken relative link	Check 11.1, runnable in-session
FM-L2	Orphan page	Reported by the crawler, never ingested; taught pages are linked at creation so they cannot become orphans
FM-L3	Scope omission	Check 11.3; <code>cite.scope_note</code> reports a page that declares no scope
FM-L4	Crawl escape	Check 11.6, on resolved paths, not on href strings
FM-L6	Superseded-content collision	Check 11.5, all five retirement steps enforced together
FM-L7	Retrieval without provenance	Provenance is a field of the hit, not a rendering choice
FM-L8	Model bypass	No mode answers without a retrieved page
FM-L9	Metadata trust failure	One parser, shared by crawler and validator
FM-L10	Local/public confusion	Every citation states <code>local</code> , with no shorter form
FM-L11	Retired-history loss	Retired pages ingested by audit; <code>:history</code> makes the capability visible

FM-L5, dynamic-page dependency, is prevented by construction: the package uses `html.parser` from the standard library and never executes JavaScript, so a page that requires rendering to be read is simply a page whose text is empty.

12. What This Does Not Claim

The console retrieving a page does not make the page true. Retrieval is a statement about where text came from, not about whether it is correct, and none of the eight checks is a fact-checker.

A taught page is a record of what somebody typed. It carries a document ID and a timestamp, which makes it look considered, and the page says on its face that it is not.

The mode controller changes what the front end does. It does not change what a language model sitting on top of the front end does. A model can read a provenance-carrying citation and paraphrase it into an unsourced claim in the next paragraph. This package can hand a model a retrieved page and can decline to hand it anything else; it cannot police what the model then says.

The `mplpb:protected` flag filters retrieval. It is not access control. The source paper's identity-gatekeeping module was left unimplemented for the same reason the paper itself states: a conversational system cannot verify who it is talking to, and implementing an inference-based gate in code would dress a stated caveat up as a mechanism.

Structure has not been shown to beat storage. That is §14.

13. Deployment

```
git clone https://github.com/<you>/smart-local-mplpb.git
cd smart-local-mplpb

python3 -m mplpb validate site      # eight checks
python3 -m mplpb chat site          # the console
python3 -m mplpb serve site         # browser view at :8000
python3 -m mplpb new mycorpus --title "Kiln Notes" --owner "Your Name"
```

No install step is required to run any of it. `pip install -e .` adds an `mplpb` command; nothing else changes. Python 3.9 or newer, standard library only, no network access at any point.

14. Open Tests and Falsifiers

Each of these would count against the design if it came back negative. None has been run.

14.1 — The ablation. Same corpus, same question set, structure removed: routing by declared scope and precedence versus flat full-text search over the same files. If flat search performs as well on answer accuracy and on correct refusals, the structure is decoration and the storage was doing the work. This is the test §14 of MPLPB-LOCAL-008 already names, applied to the front end rather than the corpus.

14.2 — Bypass rate. Give a language model this corpus through this front end, and separately as a flat directory. Measure how often it answers a corpus-specific question from its own weights rather than from a retrieved page. The console cannot bypass; the model on top of it can, and that number is the interesting one. If provenance-carrying retrieval does not lower it, FM-L8 is not addressed by this design in the way §2 claims.

14.3 — The precedence margin. The 1.5× ownership threshold is chosen, not derived. Sweep it against a corpus with genuinely adjacent spokes and measure false ties against false confident answers. If no setting separates them, the scoring model is wrong and precedence needs a different signal than rank-decayed hit weight plus declared-scope overlap.

14.4 — Does teaching survive contact with use? Run a real corpus for a month of ordinary sessions. Count taught pages that were later superseded, that were never retrieved again, and that turned out to be wrong. If

most taught pages are never read a second time, the notebook is an elaborate way of writing into a void, and the BASIC program's flat file was the right size for the job after all.

14.5 — Mode switching against real language. The implicit trigger lists are keyword sets, which is the crudest possible intent detection. Measure how often hard reality fires when it should, and how often it fires when it should not, against transcripts of ordinary questions. A mode that overrides everything is only safe if it triggers accurately; a high false-negative rate would mean the priority guarantee in §7 is nominal.

Appendix A — Command Reference

```
python3 -m mplpb chat <root> [--mode M] [--owner NAME] [--no-color]
python3 -m mplpb ask <root> "question" [--mode M]
python3 -m mplpb search <root> "query" [--status current|any] [-n N]
python3 -m mplpb id <root> DOC-ID [--history]
python3 -m mplpb crawl <root> [-o records.jsonl]
python3 -m mplpb validate <root>
python3 -m mplpb serve <root> [--host H] [--port P]
python3 -m mplpb teach <root> "question" "answer" [--owner NAME]
python3 -m mplpb new <root> [--title T] [--owner NAME] [--spoke name:scope]
```

Console commands: :help, :mode, :modes, :spokes, :where, :why, :id, :history, :teach, :revise, :status, :validate, :stats, :reload, :quit.

Appendix B — Mode Profiles as Implemented

—	strict_tool	teaching	mythic_gm	hard_reality
Priority	10	20	20	100
Narrative frame	no	no	yes	no
Scaffolded answer	no	yes	no	no
Answers without a retrieved page	no	no	no	no
Records taught pages	yes	yes	yes	no
Trigger is instruction	yes	yes	yes	no
Default retrieval status	current	current	current	current

Appendix C — Crawl Record

```
{
  "document_id": "LOCAL-SPEC-001",
  "path": "spec/link_discipline.html",
  "title": "Link Discipline – MPLPB Local",
  "category": "Specification / Architecture",
  "scope": "Relative-link rules and the crawl boundary for the local web",
  "when_to_use": "Adding a page; moving a page; a link broke",
  "updated": "2026-08-05T18:40Z v2",
}
```

```
"owner": "Mitchell D. McPhetridge",
"status": "current",
"supersedes": ["LOCAL-SPEC-OLD-001"],
"protected": false,
"spoke": "spec",
"substrate": "local",
"discovered_by": "graph",
"links": [
  { "rel": "index", "target": "index.html" },
  { "rel": "up", "target": "spec/_index.html" }
],
"text": "Extracted visible page text..."
}
```

The HTML is authoritative. The crawl record is derived.

Smart Local MPLPB is released as MIT-licensed software with CC BY 4.0 documentation. The implementation described here is a reference, not a product, and several of its claims carry tests its author records as not yet run.

MDM